Systems Track

Interview Preparation Series • Professional Edition

Concurrency & Multithreading

01

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

> **How to use this file:** Read top-to-bottom for deep understanding. Jump to §9 (Interview Questions) + §14 (Cheat Sheet) for last-minute revision.

1 Overview --- What it is

Concurrency is the ability of a system to deal with multiple tasks at the same time — tasks make *progress* simultaneously, but not necessarily *execute* at the same instant.

Parallelism is actual simultaneous execution on multiple CPU cores.

```
Concurrency (1 core):          Parallelism (2 cores):

Core 0: [T1][T2][T1][T2]       Core 0: [T1][T1][T1]
         ↑ interleaved           Core 1: [T2][T2][T2]
                                          ↑ truly simultaneous
```

Dimension	Concurrency	Parallelism
Goal	Structure / responsiveness	Speed / throughput
Requires	Scheduler, context switching	Multiple CPU cores
Example	Single-threaded event loop	Matrix multiplication on 8 cores
Risk	Race conditions, deadlocks	False sharing, synchronization

Key mental model: Concurrency is about *design*; parallelism is about *execution*. A single-core machine can be concurrent but never parallel.

A **thread** is the smallest unit of CPU scheduling — it shares the process's memory space (heap, globals) but has its own stack and program counter.

A **process** is an isolated address space; OS-level container for threads.

```
Process
├── Shared: Heap, Code, File Descriptors, Global Variables
├── Thread 1: Stack, Registers, PC
├── Thread 2: Stack, Registers, PC
└── Thread 3: Stack, Registers, PC
```

2 Why It Exists

2.1 The Hardware Reality

Modern CPUs stopped getting faster clock speeds around 2004 (the ”Power Wall”). Instead, they grew more cores. A program that uses only one thread leaves 7 of 8 cores idle. Concurrency is the answer to: *how do we exploit all this hardware?*

2.2 The Latency Reality

I/O (disk, network, DB) is 10,000–1,000,000x slower than CPU. A server that waits synchronously for each DB query handles 1 req/s instead of 10,000 req/s. Concurrency lets the CPU work on other requests while one is waiting on I/O.

```
Without concurrency:
Request 1: [CPU work]---[WAIT for DB]---[CPU work]
Request 2:                              [CPU work]---[WAIT for DB]...

With concurrency:
Request 1: [CPU work]---[WAIT]---[CPU work]
Request 2:          [CPU work]---[WAIT]---[CPU work]
                    ↑ overlap here — same total time, 2x throughput
```

2.3 Timeline of Need

- **1960s**: Batch OS needed to overlap I/O and computation
- **1990s**: Web servers needed to handle many simultaneous users
- **2004**: Multi-core CPUs made parallelism mainstream
- **2010s**: Microservices, distributed systems, reactive programming
- **Today**: GPUs, async runtimes, lock-free data structures

3 Why FAANG Cares

Company	Where Concurrency Is Critical
Google	Search index serving (billions of queries/day), Bigtable/Spanner distributed transactions, MapReduce parallelism
Meta	Thrift RPC servers handle millions of concurrent connections; News Feed ranking runs parallel ML inference pipelines
Amazon	DynamoDB uses lock-free skip lists; Lambda cold starts require efficient thread/process pool management
Apple	Grand Central Dispatch (GCD) — their concurrency framework; Core Data multi-context concurrency
Microsoft	Azure Service Bus, .NET async/await model, SQL Server lock manager, Windows thread scheduler
Uber	Real-time dispatch: matching riders/drivers under microsecond SLAs; surge pricing uses atomic counters
Databricks	Spark executor thread pools, Delta Lake transaction log with optimistic concurrency, shuffle service parallelism

Interviewers test this because:

1. Concurrency bugs are subtle, hard to reproduce, and catastrophic in production
2. Senior engineers must design concurrent systems correctly from the start
3. It reveals depth of understanding of OS, memory models, and hardware

4 Core Concepts

4.1 Threads vs Processes

Process: OS-level isolation. Memory, file handles, and address space are separate. Context switch is expensive (~1–10 µs, requires kernel mode switch, TLB flush).

Thread: Lightweight. Shares process memory. Context switch is cheaper (~100 ns). Communication via shared memory (fast but dangerous). Communication between processes requires IPC (pipes, sockets, shared memory with explicit mapping).

```
Thread Creation Cost:          Process Creation Cost:
- Stack allocation             - New address space
- Register state               - Copy page tables (or CoW)
- TCB (Thread Control Block)   - Duplicate file descriptors
≈ microseconds                 ≈ milliseconds (fork) to tens of ms
```

When to use processes: Isolation needed (security, stability — crash doesn't kill parent), different languages/runtimes. Example: Nginx worker processes.

When to use threads: Shared state needed, performance-critical (same process, shared cache), I/O overlap within one task. Example: Java web server handling requests.

4.2 Synchronization

The core problem: Multiple threads accessing shared mutable state without coordination leads to incorrect results.

```
Thread 1: READ counter (= 0)
Thread 2: READ counter (= 0)        ← both read 0!
Thread 1: counter = 0 + 1 → WRITE 1
Thread 2: counter = 0 + 1 → WRITE 1  ← lost update!
Result: counter = 1, expected: 2
```

This is a **race condition** — outcome depends on thread scheduling order.

Critical Section: Code that accesses shared state. Must be executed by only one thread at a time (mutual exclusion).

Synchronization mechanisms ranked by cost (cheap → expensive):

```
Atomic ops (CAS)  →  Spinlock  →  Mutex  →  Semaphore  →  Monitor
   ~1 ns              ~10 ns      ~25 ns      ~50 ns       ~100 ns
```

4.3 Mutex (Mutual Exclusion Lock)

A mutex is a binary lock: one thread holds it, all others block (sleep) until it's released.

Python

```python
# Python example
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    with lock:          # acquire on enter, release on exit
        counter += 1    # critical section — only 1 thread at a time

threads = [threading.Thread(target=increment) for _ in range(1000)]
for t in threads: t.start()
for t in threads: t.join()
# counter is guaranteed to be 1000
```

C++

```cpp
// C++ example
std::mutex mtx;
int counter = 0;

```

```
void increment() {
    std::lock_guard<std::mutex> lock(mtx);  // RAII — auto releases
    counter++;
}
```

Key properties:

- **Ownership:** Only the thread that acquired can release (unlike semaphore)
- **Blocking:** Waiting threads sleep (no CPU waste, but context switch cost)
- **Non-recursive by default:** Trying to re-lock from same thread → deadlock (use `recursive_mutex` if needed)

Interview trap: What happens if you forget to release a mutex? → All other threads block forever (deadlock). Always use RAII or try/finally.

4.4 Semaphore

A semaphore is a counter (initialized to N) with two operations:

- **wait() / P() / acquire():** Decrement counter. If counter < 0, block.
- **signal() / V() / release():** Increment counter. Wake one blocked thread if any.

Python

```
Binary Semaphore (N=1):  behaves like mutex BUT no ownership
Counting Semaphore (N=k): allows k threads to enter simultaneously
```

Use cases:

- **Controlling resource pool size:** N=5 → at most 5 threads use DB connection pool
- **Signaling between threads:** Producer signals, consumer waits
- **Unlike mutex:** Any thread can signal (not just the acquirer) — useful for producer/consumer

Java

```
// Java: Semaphore for connection pool
Semaphore pool = new Semaphore(10); // max 10 concurrent DB connections

void queryDB() {
    pool.acquire();     // blocks if 10 already in use
    try {
        // use DB connection
    } finally {
        pool.release(); // any thread can release
    }
}
```

4.5 Race Conditions

A race condition occurs when program correctness depends on the relative timing or ordering of concurrent operations.

Three conditions for a race:

1. Shared mutable state
2. At least one writer
3. No synchronization

Types:

- **Read-Write race:** One thread writes while another reads
- **Write-Write race:** Two threads write simultaneously
- **Check-Then-Act race:** `if (file doesn't exist) create file` — window between check and act

```java
// Classic TOCTOU (Time of Check to Time of Use) race:
if (!map.containsKey(key)) {          // Thread 1 checks: absent
    // Thread 2 also checks: absent
    map.put(key, computeValue(key));  // Thread 1 inserts
    // Thread 2 also inserts! — duplicate work or overwrite
}

// Fix: atomic putIfAbsent or synchronization
map.putIfAbsent(key, computeValue(key));
```

Heisenbugs: Race conditions that disappear when you add logging/debugging (because the act of observing changes timing). Classic in multi-threaded bugs.

4.6 Deadlock

Definition: Two or more threads permanently block each other, each waiting for a resource held by another.

```
Thread 1:  holds Lock A, waiting for Lock B
Thread 2:  holds Lock B, waiting for Lock A
           ↑ circular dependency — both block forever
```

```java
// Classic deadlock:
Lock lockA = new ReentrantLock();
Lock lockB = new ReentrantLock();

// Thread 1:
lockA.lock();
// context switch here → Thread 2 runs
lockB.lock(); // blocks! Thread 2 holds lockB

// Thread 2:
lockB.lock();
lockA.lock(); // blocks! Thread 1 holds lockA
// DEADLOCK
```

The 4 Coffman Conditions (ALL must hold for deadlock)

MHNC — Mutual exclusion, Hold-and-wait, No preemption, Circular wait

Condition	Meaning	Prevention Strategy
Mutual Exclusion	Resources can't be shared	Use shareable resources where possible
Hold-and-Wait	Thread holds resource while waiting for another	Acquire all resources at once (atomic)
No Preemption	Resources can't be forcibly taken	Allow preemption (rollback and retry)
Circular Wait	Circular chain of threads waiting	**Impose lock ordering** (most practical!)

Breaking deadlock in practice:

1. **Lock ordering:** Always acquire locks in the same global order (e.g., always lock A before B)
2. **Lock timeout:** `tryLock(timeout)` — give up and retry
3. **Deadlock detection:** Build a wait-for graph; detect cycles (Java thread dumps, database lock monitors)

```java
// Lock ordering fix:
// Both threads acquire in same order: lockA then lockB
lockA.lock();
lockB.lock();
// ... use both resources
lockB.unlock();
lockA.unlock();
// No deadlock possible — no circular wait
```

4.7 Livelock & Starvation

Livelock: Threads are not blocked but keep changing state in response to each other without making progress. Like two people in a hallway stepping left and right to avoid each other.

```
Thread 1: detects conflict → backs off → tries again
Thread 2: detects conflict → backs off → tries again
          Both backing off at the same time, forever
```

Fix: Add randomized backoff (Ethernet CSMA/CD does this).

Starvation: A thread is perpetually denied access to a resource because other threads keep getting priority. Not deadlock (others make progress), but one thread never does.

Fix: Fair queuing — FIFO ordering for lock acquisition. Java's `ReentrantLock(true)` (fair mode) guarantees FIFO.

4.8 Atomic Operations & CAS

Atomic operation: Indivisible read-modify-write. Guaranteed by hardware to complete without interruption.

Compare-And-Swap (CAS): The building block of lock-free programming.

```python
CAS(memory_location, expected_value, new_value):
    if *memory_location == expected_value:
        *memory_location = new_value
        return true  (success)
    else:
        return false (someone else changed it)
    // ALL OF THIS IS ONE ATOMIC HARDWARE INSTRUCTION (CMPXCHG on x86)
```

```java
// Java AtomicInteger uses CAS internally:
AtomicInteger counter = new AtomicInteger(0);

// Lock-free increment:
int current, next;
do {
    current = counter.get();
```

```
    next = current + 1;
} while (!counter.compareAndSet(current, next));
// Retry if another thread changed it between get() and compareAndSet()
```

ABA Problem: CAS sees value A, another thread changes A→B→A, CAS thinks nothing changed. Fix: **AtomicStampedReference** (add version number alongside value).

Lock-free vs wait-free:

- **Lock-free:** At least one thread makes progress (no global blocking), but individual threads can starve
- **Wait-free:** Every thread makes progress in bounded steps (stronger guarantee)

4.9 Memory Visibility & volatile

The hidden problem: On modern CPUs, each core has its own cache. Thread 1 writes `x = 1` to its cache; Thread 2 on another core reads `x` from its cache and sees `0`. This is a **memory visibility** problem, not just a race condition.

```
Core 0 Cache: x = 1    Core 1 Cache: x = 0    RAM: x = 0 (stale)
              ↑ write not yet flushed           ↑ Thread 2 reads stale value
```

CPU memory reordering: CPUs and compilers reorder instructions for performance. What you write in code is not necessarily executed in that order.

Java

```
// Can be reordered by compiler/CPU:
x = 1;
ready = true;  // another thread might see ready=true but x still 0!
```

volatile (Java/C++): Two guarantees:

1. **Visibility:** Write is immediately flushed to main memory; read always fetches from main memory
2. **Ordering (Java):** Prevents reordering around volatile access (happens-before guarantee)

Java

```
volatile boolean ready = false;
int x = 0;

// Writer thread:
x = 1;
ready = true;  // volatile write → x=1 guaranteed visible before ready=true

// Reader thread:
while (!ready) {}  // volatile read
System.out.println(x);  // guaranteed to print 1
```

volatile is NOT sufficient for compound operations (read-modify-write). Use `AtomicInteger` or mutex for those.

Java Memory Model (JMM) happens-before rules:

- Thread start → actions in started thread
- Lock release → subsequent lock acquire
- Volatile write → subsequent volatile read

› Thread join → actions in joined thread

4.10 Spinlock

A spinlock is a lock where the waiting thread **busy-waits** (spins in a loop) instead of sleeping.

```cpp
// Spinlock implementation:
std::atomic<bool> locked{false};

void acquire() {
    while (locked.exchange(true)) {  // CAS loop
        // spin — no sleep, no context switch
    }
}

void release() {
    locked.store(false);
}
```

Feature	Mutex	Spinlock
Waiting	Thread sleeps (OS schedules out)	Thread spins (burns CPU)
Context switch	Yes (~1-10 µs overhead)	No
Best for	Long hold times, many threads	Very short critical sections, few threads
CPU waste	No (blocked thread uses 0 CPU)	Yes (spinning = 100% CPU on core)
Use in kernel	Yes (interrupts can't sleep)	Yes

Rule of thumb: Spinlock wins if critical section is shorter than a context switch (~1 µs). Otherwise, mutex wins.

4.11 Condition Variables

Condition variables let threads wait for a specific condition to become true, releasing the associated mutex while waiting.

```python
Monitor pattern = Mutex + Condition Variable
```

```java
// Java: Producer-Consumer with condition variables
import java.util.concurrent.locks.*;

Lock lock = new ReentrantLock();
Condition notFull  = lock.newCondition();
Condition notEmpty = lock.newCondition();
Queue<Integer> queue = new LinkedList<>();
int CAPACITY = 10;

void produce(int item) throws InterruptedException {
    lock.lock();
    try {
        while (queue.size() == CAPACITY)
            notFull.await();       // releases lock, waits
```

```
        queue.add(item);
        notEmpty.signal();          // wake one consumer
    } finally {
        lock.unlock();
    }
}

void consume() throws InterruptedException {
    lock.lock();
    try {
        while (queue.isEmpty())
            notEmpty.await();       // releases lock, waits
        int item = queue.poll();
        notFull.signal();           // wake one producer
        return item;
    } finally {
        lock.unlock();
    }
}
```

Critical: Always use `while` (not `if`) when checking condition after await. Spurious wake-ups (thread wakes without being signaled — real OS phenomenon) and multiple waiters can cause the condition to be false again by the time you execute.

4.12 Producer-Consumer Problem

Classic synchronization problem. Bounded buffer shared between producers (add items) and consumers (remove items).

Constraints:

1. Producer must not add to full buffer
2. Consumer must not remove from empty buffer
3. Only one thread accesses buffer at a time

Python

```python
# Python: Producer-Consumer with semaphores
import threading, time, random

CAPACITY = 5
mutex = threading.Semaphore(1)   # binary — protects buffer access
empty = threading.Semaphore(CAPACITY)  # slots available to produce
full  = threading.Semaphore(0)         # items available to consume
buffer = []

def producer():
    for i in range(10):
        item = random.randint(1, 100)
        empty.acquire()      # wait for empty slot
        mutex.acquire()      # enter critical section
        buffer.append(item)
        print(f"Produced {item}, buffer={buffer}")
        mutex.release()      # exit critical section
        full.release()       # signal: one more item available

def consumer():
    for i in range(10):
        full.acquire()       # wait for item
        mutex.acquire()
```

```
        item = buffer.pop(0)
        print(f"Consumed {item}, buffer={buffer}")
        mutex.release()
        empty.release()      # signal: one more slot available

t1 = threading.Thread(target=producer)
t2 = threading.Thread(target=consumer)
t1.start(); t2.start()
t1.join(); t2.join()
```

Key insight: The ORDER of semaphore operations matters.

- Producer: `empty.acquire()` THEN `mutex.acquire()` (wrong order → deadlock with full buffer)
- Consumer: `full.acquire()` THEN `mutex.acquire()`

4.13 Reader-Writer Problem

Multiple readers can read simultaneously (no conflict). Writers need exclusive access.

Priority variants:

- **Reader priority:** Readers never wait if any reader is active. Writers can starve.
- **Writer priority:** New readers wait if a writer is waiting. Fairer.

Java

```java
// Java ReadWriteLock — production-quality solution
ReadWriteLock rwLock = new ReentrantReadWriteLock();
Lock readLock  = rwLock.readLock();
Lock writeLock = rwLock.writeLock();

// Multiple readers can hold simultaneously:
void read() {
    readLock.lock();
    try { /* read shared data */ }
    finally { readLock.unlock(); }
}

// Only one writer, and only when no readers:
void write() {
    writeLock.lock();
    try { /* modify shared data */ }
    finally { writeLock.unlock(); }
}
```

State machine:

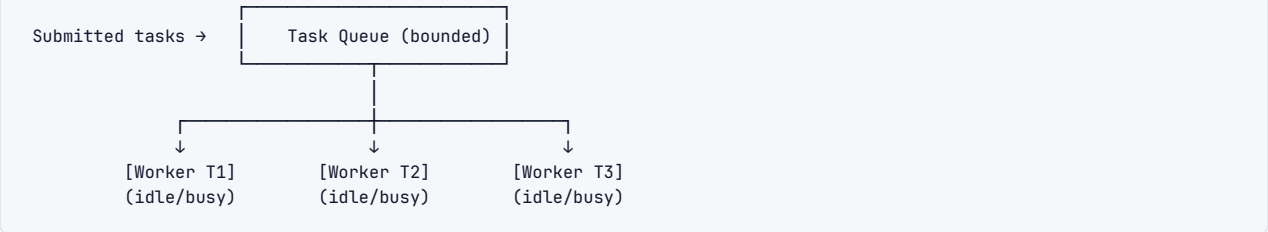
```
State: UNLOCKED
  → readLock()  → READING (multiple readers OK)
  → writeLock() → WRITING (exclusive)

State: READING (N readers)
  → readLock()   → READING (N+1 readers)
  → writeLock()  → BLOCKS until all readers done
  → readUnlock() → READING (N-1), or UNLOCKED if N=1

State: WRITING
  → readLock()  → BLOCKS
  → writeLock() → BLOCKS
  → writeUnlock() → UNLOCKED
```

Interview insight: ReadWriteLock only wins when reads are dominant (>90%). With heavy

writes, the overhead of tracking readers/writers makes it slower than a plain mutex.

4.14 Thread Pools

Creating threads is expensive. Thread pools maintain a set of pre-created worker threads that pick up tasks from a queue.



ThreadPoolExecutor parameters (Java):

```java
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    4,          // corePoolSize: always-on threads
    8,          // maximumPoolSize: max threads when queue full
    60L,        // keepAliveTime: idle non-core threads die after this
    TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(100),  // bounded task queue
    new ThreadPoolExecutor.CallerRunsPolicy()  // rejection policy
);
```

Rejection policies when queue full + max threads reached:

- `AbortPolicy` (default): throw `RejectedExecutionException`
- `CallerRunsPolicy`: caller thread runs the task (backpressure)
- `DiscardPolicy`: silently drop the task
- `DiscardOldestPolicy`: drop oldest queued task, retry

Pool sizing formula:

```
CPU-bound tasks:   threads ≈ CPU_cores + 1 (extra for occasional blocking)
I/O-bound tasks:   threads ≈ CPU_cores × (1 + wait_time/compute_time)
Example: 8 cores, 90% I/O wait → 8 × (1 + 9) = 80 threads
```

4.15 False Sharing

Two threads modify different variables that happen to sit in the **same cache line** (64 bytes on x86). Each core's write invalidates the other core's cache line, causing constant cache misses — as slow as sharing a variable, but no actual sharing.

```
Cache line (64 bytes):
[counter_A (8 bytes)][counter_B (8 bytes)][... padding ...]
     ↑ Thread 1            ↑ Thread 2
     writes here           writes here
     → invalidates Thread 2's copy of ENTIRE cache line
     → Thread 2 must reload from memory
     → thrashing even though they access different variables!
```

```java
// Fix: padding to separate variables into different cache lines
class PaddedCounter {
    long p1, p2, p3, p4, p5, p6, p7; // 56 bytes padding
    volatile long value;              // 8 bytes — own cache line
    long q1, q2, q3, q4, q5, q6, q7; // 56 bytes padding
}
// Or use @Contended annotation in Java 8+ (JEP 142)
```

LMAX Disruptor — high-performance ring buffer — was designed specifically to eliminate false sharing.

4.16 Python GIL (Global Interpreter Lock)

CPython (the standard Python interpreter) has a global mutex — the GIL — that only allows one thread to execute Python bytecode at a time.

```
Thread 1: [Python bytecode]─┐
Thread 2:                   └─ GIL allows only one at a time
Thread 3: [C extension]──────── (bypasses GIL — can run in parallel!)
```

Implications:

- Python threads don't give CPU parallelism for CPU-bound work
- Threads still help for I/O-bound work (GIL released during I/O)
- For CPU parallelism: use `multiprocessing` (separate processes, each with own GIL) or C extensions (NumPy, TensorFlow bypass the GIL)

```python
# CPU-bound: threads DON'T help (GIL bottleneck)
import threading
# 4 threads still run sequentially for pure Python computation

# CPU-bound: use multiprocessing
from multiprocessing import Pool
with Pool(4) as p:
    results = p.map(cpu_heavy_fn, data)  # true parallelism

# I/O-bound: threads work fine
import threading
# threads waiting on network/disk release GIL → true concurrency
```

Python 3.13+: "No-GIL" build (PEP 703) is in experimental phase, using fine-grained per-object locking instead.

4.17 Async vs Threads

Both deal with concurrency, but with different execution models.

```python
Threads:                          Async (coroutines):
OS schedules preemptively         Cooperative — yields explicitly
Each thread has own stack (MB)    One stack shared, tiny frames (KB)
Context switch: OS interrupt      Context switch: await keyword
Good for: CPU-bound, blocking C libs  Good for: I/O-bound, many connections

```

```
Node.js/Python asyncio = 1 thread, event loop, callbacks/coroutines
Java Virtual Threads (Project Loom) = async benefits with thread API
```

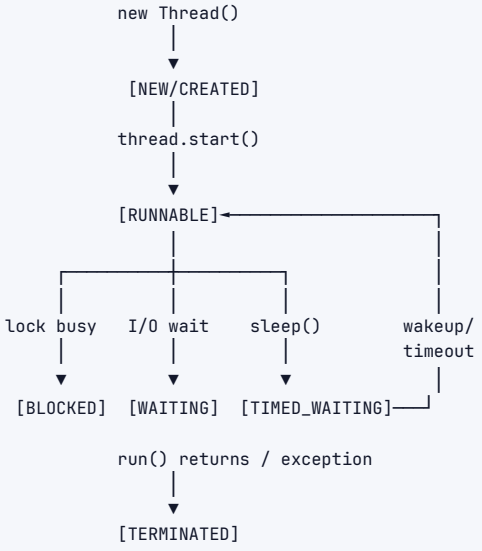
```python
# Async Python: 10,000 concurrent HTTP requests, 1 thread
import asyncio, aiohttp

async def fetch(session, url):
    async with session.get(url) as response:
        return await response.text()

async def main():
    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        results = await asyncio.gather(*tasks)
```

The fundamental difference:

```
Threads: preemptive multitasking — OS can switch anytime
Async:   cooperative multitasking — switch only at await points
         → no race conditions between await points (within one coroutine)
         → but: long-running sync code blocks the entire event loop
```

5 Architecture / Diagrams

5.1 Thread Lifecycle



5.2 Mutex vs Semaphore vs Monitor

	Mutex	Semaphore
Value	binary (0 or 1)	integer (0 to N)

```
| Ownership    | yes (locked thread   | no (any thread can signal) |
|              | must unlock)         |                            |
| Use case     | mutual exclusion     | signaling, resource count  |
| Recursive?   | no (by default)      | yes (counter-based)        |
| Priority     | can do inheritance   | typically not              |
└──────────────┴──────────────────────┴────────────────────────────┘

Monitor = Object/Class with:
  - Internal mutex (one thread at a time)
  - Condition variables (wait/notify)
  Java's synchronized methods/blocks = Monitor
```

5.3 Deadlock Detection: Wait-For Graph

```
T1 ──waits──► T2
▲             │
│             waits
└──waits── T3 ◄┘

Cycle T1→T2→T3→T1 = DEADLOCK
```

5.4 Lock Ordering to Prevent Deadlock

```
Global lock order: L1 < L2 < L3 < L4

Thread A needs L2, L4:   acquire L2 → acquire L4 → release L4 → release L2
Thread B needs L1, L4:   acquire L1 → acquire L4 → release L4 → release L1
Thread C needs L2, L3:   acquire L2 → acquire L3 → release L3 → release L2

No circular dependency possible — all follow same ordering
```

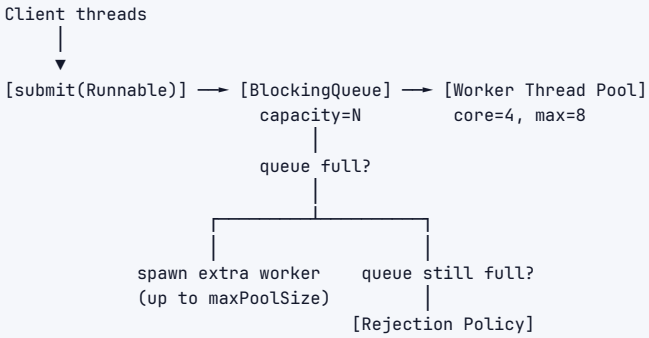
5.5 CAS Operation Internals

```python
CMPXCHG instruction (x86):
1. Read value from memory address
2. Compare with expected
3. If equal: write new value, return true
4. If not equal: return false (read actual value)
Steps 1-4 are ATOMIC — cannot be interrupted by another CPU
```

5.6 Memory Model: Visibility Issue

```
Multi-core CPU:

Core 0                    Core 1
┌──────┐                  ┌──────┐
│ L1 $ │ x=1 (dirty)      │ L1 $ │ x=0 (stale)
└──────┘                  └──────┘
   │                         │
┌──▼─────────────────────────▼──┐
│        L3 Cache / RAM         │  x=0 (not yet flushed)
└───────────────────────────────┘

volatile/memory fence forces:
- Write: flush to RAM immediately
- Read: invalidate local cache, read from RAM
```

5.7 Thread Pool Flow



6 Real-World Examples

6.1 Google: Bigtable Tablet Server

Bigtable tablet servers use read-write locks to protect tablet metadata. Multiple reads (serving queries) proceed concurrently; mutations (compactions, splits) acquire write lock exclusively. This allows high read throughput while ensuring consistency during structural changes.

6.2 Meta: Folly's Concurrency Primitives

Meta's open-source Folly library (`folly::Synchronized<T>`) wraps any data structure with a mutex automatically. Their `SharedMutex` is a high-performance reader-writer lock used in Thrift servers where millions of RPC calls read configuration but writes are rare.

6.3 Amazon DynamoDB: Lock-Free Data Structures

DynamoDB uses lock-free concurrent skip lists for its in-memory data structures. CAS operations handle concurrent inserts/deletes without traditional locking, achieving high throughput at low latency for partition-local operations.

6.4 Uber: Atomic Counters for Surge Pricing

Uber's surge pricing uses `AtomicLong` counters to track active trips per geohash cell. When a driver completes a trip, `decrementAndGet()` happens atomically across multiple dispatch servers using CAS — no locks needed for this critical counting operation.

6.5 Linux Kernel: RCU (Read-Copy-Update)

RCU is an extreme read-write lock optimization: readers never lock at all (zero overhead). Writers make a copy, update it, then atomically swap the pointer. Old copy is freed when no readers reference it. Used in Linux networking, file system paths — thousands of reads per microsecond.

6.6 Java's ConcurrentHashMap

Pre-Java 8: one lock per segment (16 segments = 16-way parallelism). Java 8+: CAS for `put` when bucket is empty; per-bucket `synchronized` when needed. Readers never block. This gives near-linear scalability with thread count for most workloads.

7 Real-Life Analogies

One kitchen, one brigade — every concept is a cook, a tool, or a station in the same busy restaurant.

Concept	Analogy — *the restaurant kitchen*
Thread	A line cook at a station — shares the kitchen (process) but works their own tickets and prep (stack)
Process	A separate restaurant down the street — its own kitchen and pantry; the two only talk by phone (IPC)
Mutex	The single razor-sharp chef's knife — only one cook can hold it; the rest wait their turn to chop
Semaphore	The stove's 4 burners — at most 4 dishes cook at once; finish one and the next cook claims the burner
Deadlock	Cook A holds the only pan and needs the oven; Cook B holds the oven and needs the pan — both freeze
Livelock	Two cooks meet in the narrow doorway and keep stepping the same way to yield — polite, but nobody passes
Starvation	The new cook who's always skipped for the busy grill — everyone else keeps getting picked first
Race Condition	Two cooks grab the last egg for different orders — one ticket goes out wrong
Spinlock	A cook hovering at the oven, opening it every few seconds to check if it's free instead of prepping
Condition Variable	The expo bell — cooks rest until ”order up!” rings, then wake and plate
volatile	The shared specials whiteboard — everyone reads the same board instantly, not private notepads
CAS	”Is table 5's ticket still unclaimed?” — glance at the rail before grabbing it; if taken, try the next
Thread Pool	A fixed brigade pulling tickets off the rail — idle cooks wait, finished cooks grab the next ticket
False Sharing	Two cooks elbowing over one cutting board for different vegetables — both slow down without truly sharing
ABA Problem	A burner looked free; a cook used and cleared it while you blinked — you assume nothing changed, get burned
Producer-Consumer	Chefs plate onto the pass shelf (limited space); waiters carry out — chefs pause when full, waiters wait when empty
Reader-Writer	The recipe board — any number of cooks read at once, but the head chef locks it alone to rewrite a recipe
GIL	A tiny kitchen with a single stove — hire ten cooks, but only one can actually cook at any moment
Async/Await	One waiter working ten tables — takes the next order while the kitchen cooks the last, never idle

8 Memory Tricks / Mnemonics

8.1 Deadlock 4 Conditions: "MHNC" (Make Him Never Circulate)

- **M**utual Exclusion
- **H**old and Wait
- **N**o Preemption

- **C**ircular Wait

Break any ONE to prevent deadlock:

- Break M: Use shareable resources (read-only data)
- Break H: Acquire all locks atomically at once
- Break N: Allow lock preemption (databases do this with rollback)
- Break C: **LOCK ORDERING** — most practical solution in software

8.2 Semaphore P/V: "P = Post office, V = Vacate"

- **P** (proberen/test) = acquire, wait, decrement → "waiting in POST office queue"
- **V** (verhogen/increment) = release, signal → "VACATE the window, next!"

8.3 Thread States: "NEW RABBIT WAITS BUSILY THEN DIES"

- NEW → RUNNABLE → WAITING/TIMED_WAITING/BLOCKED → TERMINATED

8.4 Happens-Before Rules (Java): "SLVJ" --- Start, Lock, Volatile, Join

- **S**tart: thread.start() happens-before thread's actions
- **L**ock: unlock happens-before subsequent lock
- **V**olatile: write happens-before subsequent read
- **J**oin: thread actions happen-before join() returns

8.5 CAS vs Lock: "CAS for counters, Lock for compounds"

- Single variable update → CAS/Atomic
- Multiple variables must change together → Lock

8.6 Pool Sizing: "I/O = Cores × 10, CPU = Cores + 1"

- Pure CPU work: `cores + 1` threads
- Pure I/O work: `cores × 10` threads (rough rule)
- Mixed: measure and tune with load testing

8.7 False Sharing Fix: "Pad to 64"

- Cache line = 64 bytes on x86
- Pad hot variables so they don't share a cache line

8.8 volatile vs synchronized:

- **volatile:** single-variable visibility + ordering (no atomicity for compound ops)
- **synchronized:** visibility + ordering + atomicity for critical section

8.9 Memory model mental model: "Write → Fence → Read"

- volatile write = release fence (all prior writes visible after)
- volatile read = acquire fence (all subsequent reads see previous writes)

9 Common Interview Questions

9.1 Q1: What is a race condition? How do you prevent it?

Model answer: A race condition occurs when program output depends on relative thread scheduling. Prevents: identify shared mutable state, protect with mutex/synchronized, or use lock-free atomics, or eliminate sharing (immutability, thread-local storage).

Follow-ups:

- ”Give me an example of a race condition in a real system” → Double-checked locking without volatile in pre-Java 5 singleton
- ”Can you have a race condition with a single CPU?” → Yes, due to context switches between instructions

9.2 Q2: Explain the four conditions for deadlock and how to prevent each.

Model answer: MHNC — Mutual Exclusion, Hold-and-Wait, No Preemption, Circular Wait. Prevention: lock ordering (breaks circular wait), acquire all locks at once (breaks hold-and-wait), tryLock with timeout (breaks no-preemption).

Follow-ups:

- ”How does a database detect deadlock?” → Wait-for graph cycle detection; victim selection by transaction age/cost
- ”What's the difference between deadlock prevention and avoidance?” → Prevention breaks a condition; avoidance (Banker's algorithm) dynamically refuses unsafe states

9.3 Q3: Implement producer-consumer with two semaphores.

Model answer: (See code in §Producer-Consumer section above)

Key points to mention:

- `empty` semaphore initialized to buffer capacity
- `full` semaphore initialized to 0
- Acquire resource semaphore BEFORE mutex to avoid deadlock
- Why while instead of if for condition check

Follow-ups:

- ”What if we have 3 producers and 5 consumers?” → Same code, semaphores handle N:M automatically
- ”How would you implement this without semaphores?” → Mutex + condition variables (as in Java example)

9.4 Q4: What is the difference between a mutex and a semaphore?

Model answer:

- Mutex: binary, has ownership (acquirer must release), used for mutual exclusion
- Semaphore: counter 0-N, no ownership (any thread can signal), used for resource counting and signaling

Follow-up: ”Can you implement a mutex with a semaphore?” → Yes, binary semaphore with convention that only acquirer calls signal. But you lose deadlock detection and priority inheritance benefits.

9.5 Q5: Explain Compare-And-Swap and the ABA problem.

Model answer: CAS atomically: reads value, compares to expected, writes new value if match, returns success/fail. Single hardware instruction (CMPXCHG). ABA: value changed A→B→A between your read and CAS — CAS succeeds thinking nothing changed. Fix: add version counter (`AtomicStampedReference`).

Follow-up: ”Where would ABA actually cause a bug?” → Lock-free linked list: node removed, memory freed, new node allocated at same address. CAS sees same pointer, thinks old node still there → use-after-free or corruption.

9.6 Q6: What is false sharing and how do you fix it?

Model answer: Two threads write to different variables sharing a 64-byte cache line. Each write invalidates the other core's cache line, causing cache misses even though the data is logically independent. Fix: pad struct so hot variables occupy their own cache line.

Follow-up: "How do you detect false sharing?" → `perf stat -e cache-misses` on Linux, Intel VTune "false sharing" analysis. Look for high L1/L2 cache miss rates with no logical sharing.

9.7 Q7: What is the Python GIL and when does it matter?

Model answer: Global Interpreter Lock — CPython mutex preventing multiple native threads from executing Python bytecode simultaneously. Matters for CPU-bound parallelism (threads don't help). Doesn't matter for I/O-bound concurrency (GIL released during I/O, threads work fine). Fix for CPU-bound: `multiprocessing`, C extensions, or PyPy.

Follow-up: "Is the GIL being removed?" → Python 3.13 experimental no-GIL build (PEP 703). Not default yet. Uses per-object reference counting with atomic ops instead.

9.8 Q8: Explain volatile in Java. When is it sufficient?

Model answer: Volatile ensures visibility (writes immediately visible to other threads via memory barrier) and ordering (prevents reordering around the volatile access). Sufficient for: flag variables, single-variable state published once. NOT sufficient for: `counter++` (read-modify-write is not atomic). Use `AtomicInteger` for that.

Follow-up: "What's the difference between volatile and synchronized?" → synchronized adds atomicity for compound actions + mutual exclusion. volatile is lighter but no mutual exclusion.

9.9 Q9: How does Java's ReadWriteLock work, and when would you not use it?

Model answer: Allows multiple concurrent readers OR one exclusive writer. Writer blocks when readers active; readers block when writer active. Avoids when: write-heavy workloads (overhead > benefit), very short critical sections (overhead of tracking readers), when you need upgrade from read to write lock (causes deadlock — not supported).

Follow-up: "What's StampedLock?" → Java 8 addition. Adds optimistic reading: try a read without lock, check stamp at end; if stamp changed, fall back to real read lock. Even better read throughput when conflicts rare.

9.10 Q10: Design a thread-safe singleton in Java.

Model answer (multiple approaches):

Java

```java
// Approach 1: Eager initialization (simplest, safe)
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    public static Singleton getInstance() { return INSTANCE; }
}

// Approach 2: Double-checked locking (lazy, safe in Java 5+)
public class Singleton {
    private static volatile Singleton instance;  // volatile REQUIRED
    public static Singleton getInstance() {
        if (instance == null) {                  // first check (no lock)
            synchronized (Singleton.class) {
                if (instance == null) {          // second check (with lock)
                    instance = new Singleton();
                }
```

```
            }
        }
        return instance;
    }
}

// Approach 3: Initialization-on-demand holder (best — lazy + safe)
public class Singleton {
    private static class Holder {
        static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() { return Holder.INSTANCE; }
}
```

Why volatile in DCL? Without volatile, another thread may see partially constructed object (constructor writes reordered to after `instance = new Singleton()` write).

9.11 Q11: What is a condition variable and why use `while` instead of `if`?

Model answer: Condition variable pairs with a mutex. Thread calls `wait()` to atomically release mutex and sleep until signaled. After waking, re-acquires mutex. Use `while` because: (1) spurious wakeups (real POSIX behavior — thread wakes without explicit signal), (2) multiple waiters — condition may be true when signaled but false again by the time this thread executes.

9.12 Q12: Difference between thread pool's fixed vs cached vs work-stealing executors?

```
Executors.newFixedThreadPool(n):   fixed n threads, unbounded queue
                                   → predictable resource usage, can queue forever
Executors.newCachedThreadPool():   0 to Integer.MAX threads, 60s TTL
                                   → good for short-lived tasks, can explode memory
Executors.newWorkStealingPool():   ForkJoinPool, work-stealing deques per thread
                                   → best for recursive/tree tasks (divide and conquer)
```

10 Senior-Level Discussion Points

10.1 1. Amdahl's Law --- Theoretical Speedup Limit

```
Speedup = 1 / (S + (1-S)/N)

S = serial fraction of program
N = number of processors

If 5% of your code must be serial:
  N=∞ → max speedup = 1/0.05 = 20x (even with infinite cores!)
```

Implication: Identify and minimize serial bottlenecks before adding more threads/cores.

10.2 2. Memory Model Sophistication

Modern CPUs use TSO (Total Store Order — x86), allowing stores to be reordered relative to other stores from other cores' perspective. ARM/POWER allow more reorderings. High-performance code targeting multiple architectures must use explicit memory fences (`std::atomic_thread_fence`), not just volatile.

10.3 3. Lock-Free Algorithm Design Challenges

Beyond ABA: lock-free algorithms are notoriously difficult. Even with CAS, you need careful thought about:

- Memory reclamation (when is it safe to free a node seen by other threads?)
- Progress guarantees (lock-free vs wait-free)
- Platform memory models

Epoch-based reclamation and hazard pointers are standard techniques for safe memory reclamation in lock-free data structures.

10.4 4. Virtual Threads (Java 21 Project Loom)

Java 21 introduces virtual threads — millions of lightweight threads mapped to OS threads by the JVM. Blocking operations (I/O, sleep) on virtual threads park the virtual thread without blocking the underlying OS thread. This combines the programming simplicity of threads with async scalability.

```java
// Before Loom: thread-per-request limited to ~10K concurrent requests
// With Loom: 1M virtual threads, each blocking synchronously
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000_000; i++) {
        executor.submit(() -> {
            Thread.sleep(1000);  // blocks virtual thread, not OS thread
            return doWork();
        });
    }
}
```

10.5 5. STM (Software Transactional Memory)

Database-like transactions for memory: group reads/writes into a transaction, auto-retry on conflict. Clojure uses STM for its reference types. Eliminates deadlocks (transactions can always roll back) but high overhead for write-heavy workloads.

10.6 6. Reactive/Actor Model

Actor model (Akka, Erlang): Each actor has mailbox and processes one message at a time — no shared state. Concurrency via message passing. Deadlock-resistant by design (no locks). Actors can be distributed transparently.

10.7 7. Lock Granularity Trade-offs

```
Coarse-grained locking:    One lock for entire data structure
                           → Simple, but poor concurrency
Fine-grained locking:      Lock per node/segment
                           → Complex, good concurrency, risk of deadlock
Lock-free:                 CAS-based, no locks
                           → Best throughput, very complex to implement correctly
```

ConcurrentHashMap progression: Per-structure lock → 16 segments → per-bucket CAS/sync (Java 8+).

11 Typical Mistakes Candidates Make

11.1 1. Using `if` instead of `while` for condition variable waits

Symptom: Intermittent NullPointerExceptions or incorrect values after wait(). **Fix:** Always `while (condition not met) { wait(); }`

11.2 2. Wrong semaphore acquire order causing deadlock

Symptom: Producer-consumer deadlocks when buffer is full. **Fix:** Acquire resource semaphore (empty/full) before mutex, never after.

11.3 3. Checking for == `null` on volatile reference without synchronized block

Symptom: Two instances created in double-checked locking without volatile. **Fix:** Declare instance variable as volatile.

11.4 4. Forgetting to release locks (no RAII/try-finally)

Symptom: System hangs intermittently when exceptions thrown in critical section. **Fix:** Always use try-finally or RAII (`lock_guard`, `with lock:`).

11.5 5. Using Thread.stop() or Thread.suspend() (deprecated)

Symptom: Orphaned locks, corrupt state. **Fix:** Use interrupt flags + cooperative checking (`while (!Thread.interrupted())`).

11.6 6. Thinking volatile fixes race conditions for compound operations

Symptom: Lost updates on `volatile int counter` with `counter++`. **Fix:** Use `AtomicInteger.incrementAndGet()`.

11.7 7. Assuming HashMap.get() is thread-safe because it's "read-only"

Symptom: Infinite loops or NPE during concurrent modification (HashMap can infinite loop on get during rehash in Java). **Fix:** Use `ConcurrentHashMap`.

11.8 8. Ignoring lock ordering → deadlock in production

Symptom: Rare deadlocks that only happen under load. **Fix:** Establish and document global lock acquisition order.

11.9 9. Creating too many threads for CPU-bound work

Symptom: Performance degrades with 100 threads for CPU work on 8-core machine. **Fix:** Thread count ≈ core count for CPU-bound.

11.10 10. Assuming thread-safe classes compose safely

Symptom: `if (!list.contains(x)) list.add(x)` on a `CopyOnWriteArrayList` still has race. **Fix:** The compound check-then-act still needs external synchronization.

12 How This Connects To Other Topics

12.1 Operating Systems

- Thread scheduling: preemptive vs cooperative, priority inversion
- Kernel mutexes vs userspace futexes (Linux `futex` syscall — fast path is CAS in userspace, slow path escalates to kernel)
- Context switching cost → design trade-offs for thread pool sizing
- Virtual memory and copy-on-write (fork() performance)

12.2 System Design

- Stateless services avoid shared mutable state — scale horizontally without concurrency bugs
- Connection pools (JDBC, Redis clients) use semaphores for resource limiting
- Message queues (Kafka, SQS) implement producer-consumer at distributed scale
- Distributed locks (Redis `SET NX EX`, ZooKeeper) = distributed mutex
- Optimistic locking (version numbers in DB) = CAS at database level
- MVCC (Multi-Version Concurrency Control) = database reader-writer problem solution

12.3 Performance Engineering

- False sharing → memory-conscious data layout (LMAX Disruptor pattern)
- Lock contention profiling (Java VisualVM, Linux `perf lock`)
- Amdahl's Law limits scaling benefits
- Cache-coherent NUMA architectures — prefer thread-local data

12.4 Databases

- ACID transactions: serializable isolation = "writer-writer exclusion + reader-writer consistency"
- Pessimistic locking (`SELECT FOR UPDATE`) = mutex
- Optimistic locking (version field) = CAS
- Deadlock detection via wait-for graph — databases kill the youngest transaction
- MVCC: reads don't block writes, writes don't block reads — read-committed, snapshot isolation

12.5 Distributed Systems

- CAP theorem: consistency requires cross-node synchronization (distributed mutex)
- Consensus algorithms (Raft, Paxos) = distributed condition variable for leader election
- Distributed transactions (2PC) → distributed deadlock possible
- Event sourcing / CQRS → separates reads and writes to avoid reader-writer contention

13 FAANG Interview Tips

13.1 Strategy by Interview Type

Coding round: If given a multi-threading problem:

1. First identify shared state
2. Show you understand the race condition before writing fix
3. Start with correct-but-simple (mutex), then optimize if asked
4. Always mention: "I'd use `while` not `if` for wait loops"

System design round:

- Proactively mention concurrency when designing: "The rating service has a counter — I'd use atomic increment to avoid locks"
- Show awareness of pool sizing: "For I/O-heavy service, I'd size thread pool larger than CPU cores"
- Discuss connection pool limits, semaphores for rate limiting

Behavioral/technical deep-dive:

- Have a story about debugging a race condition or deadlock (even in side projects)

- Mention specific tools: Java thread dumps, `jstack`, Go's race detector (`go test -race`), Helgrind

13.2 Language-Specific Tips

Java: Know `synchronized`, `volatile`, `java.util.concurrent.*`, `ReentrantLock`, `ReadWriteLock`, `AtomicInteger`, `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `CompletableFuture`. Virtual threads (Java 21).

Python: Know the GIL, `threading.Lock()`, `threading.Condition()`, `multiprocessing` for CPU parallelism, `asyncio` for I/O concurrency, `concurrent.futures`.

C++: `std::mutex`, `std::lock_guard`, `std::unique_lock`, `std::condition_variable`, `std::atomic`, `std::memory_order_*`, `std::thread`.

Go: Goroutines, channels (preferred over shared memory), `sync.Mutex`, `sync.RWMutex`, `sync.WaitGroup`, `sync/atomic`, `go test -race`.

13.3 Common FAANG Patterns They Test

1. **Thread-safe singleton** (almost every Java interview)
2. **Producer-consumer with bounded buffer** (Amazon, Google favorites)
3. **Rate limiter using semaphore** (Uber, Meta)
4. **ReadWriteLock for cache** (high-read, rare-write)
5. **Parallel merge sort / parallel tree traversal** (divide and conquer + thread pool)
6. **Deadlock diagnosis from thread dump** (senior-level Google SRE/SWE)

13.4 What Separates Good from Great Answers

Good Answer	Great Answer
"Use a mutex"	"Mutex here, but if reads dominate, ReadWriteLock improves throughput"
"volatile makes it thread-safe"	"volatile ensures visibility and ordering but not compound atomicity"
"Threads run concurrently"	"On N cores, up to N threads run in parallel; others context-switch"
"Use thread pool"	"I/O-bound → size at cores×(1+wait/compute); CPU-bound → cores+1"
"Deadlock: two threads waiting"	"Four Coffman conditions; break circular wait via lock ordering"

14 Revision Cheat Sheet

14.1 10-Minute Revision Summary

The 3 fundamental problems:

1. **Race conditions** → shared mutable state + no sync → use locks or atomics
2. **Deadlocks** → MHNC conditions → break circular wait with lock ordering
3. **Visibility** → CPU caches → volatile for flags, synchronized/atomic for compounds

Key primitives:

- **Mutex** = binary lock with ownership, one thread at a time
- **Semaphore** = counter lock, no ownership, N threads at a time
- **Condition variable** = wait for condition (ALWAYS while-loop, not if)
- **volatile** = visibility + ordering, NOT atomicity for compounds

- **CAS/Atomic** = lock-free compare-and-swap, handles ABA with stamp
- **SpinLock** = CPU-burning wait, use only for < 1µs critical sections

Key algorithms:

- **Producer-Consumer:** empty/full semaphores + mutex; acquire resource semaphore BEFORE mutex
- **Reader-Writer:** ReadWriteLock; readers can share; writer is exclusive
- **Thread Pool:** queue + N workers; size = cores+1 (CPU) or cores×10 (I/O)

Language facts:

- **Python:** GIL = only 1 thread executes Python bytecode; use multiprocessing for CPU parallelism
- **Java:** `synchronized` = monitor (mutex + condition); `volatile` = visibility; `Atomic*` = CAS
- **Go:** prefer channels over shared memory; goroutines are cheap (~2KB stack)

14.2 Key Points to Remember

- Concurrency = dealing with many things at once (design); Parallelism = doing many things at once (execution)
- Thread creation cost: µs; Context switch cost: 100ns (thread) vs µs (process)
- Deadlock 4 conditions: **MHNC** — break ANY ONE to prevent
- Lock ordering is the most practical deadlock prevention in software
- Always `while`, never `if`, when checking wait condition
- Acquire resource semaphore BEFORE mutex in producer-consumer
- volatile alone is insufficient for `count++` — use AtomicInteger
- False sharing: pad to 64 bytes to separate hot variables into own cache lines
- Pool sizing: CPU-bound → N+1; I/O-bound → N×(1 + wait/compute)
- Python GIL: threads for I/O, multiprocessing for CPU
- Livelock: both active, no progress; Starvation: one thread never gets resource

14.3 Compact Cheat Sheet Table

Concept	One-line Definition	Key Code / Formula	Gotcha
Thread	Lightweight execution unit, shared memory	`new Thread(runnable).start()`	Stack is separate; heap is shared
Process	Isolated address space	`fork()` / `ProcessBuilder`	IPC needed for communication
Mutex	Binary lock with ownership	`lock.acquire(); ... lock.release()`	Forgetting release = deadlock
Semaphore	Counter lock, any thread signals	`sem.wait(); ... sem.signal()`	Acquire resource sem before mutex
Condition Var	Sleep + release lock atomically	`while(!cond) cv.wait(lock)`	Use while, not if
volatile	Visibility + ordering, no atomicity	`volatile boolean ready`	Not enough for `count++`
CAS / Atomic	Hardware read-modify-write	`AtomicInt.compareAndSet(old, new)`	ABA problem
Spinlock	Busy-wait loop	`while(lock.test_and_set()) {}`	Wastes CPU; only for short waits
Deadlock	Circular wait for resources	4 conditions: MHNC	Prevent: global lock ordering

Concept	One-line Definition	Key Code / Formula	Gotcha
Livelock	Active but no progress	Randomized backoff	Looks like progress, isn't
Starvation	One thread perpetually denied	Fair queue (FIFO lock)	Different from deadlock
Thread Pool	Pre-created workers + task queue	cores+1 (CPU), cores×10 (I/O)	Unbounded queue = memory leak
False Sharing	Different vars, same cache line	Pad to 64 bytes	Hard to detect; use perf tools
Python GIL	One thread at a time in CPython	Use multiprocessing for CPU work	I/O threads still help
ReadWriteLock	Multiple readers OR one writer	`rwLock.readLock()`	Not worth it for write-heavy
Producer-Consumer	Bounded buffer sync problem	empty + full semaphores + mutex	Order: resource sem → mutex
async/await	Cooperative coroutines on event loop	`await asyncio.gather(*tasks)`	Sync code blocks entire loop

14.4 Most Important Interview Concepts (Ranked by Frequency)

1. **Race condition definition + example + fix** — asked everywhere
2. **Deadlock: 4 conditions (MHNC) + prevention** — almost guaranteed at senior level
3. **Producer-Consumer implementation** — Amazon, Google standard question
4. **volatile vs synchronized / atomic** — Java-focused companies
5. **Thread pool sizing** — system design integration
6. **mutex vs semaphore** — definitional but important
7. **False sharing** — performance-focused companies (Databricks, Meta)
8. **Condition variable with while-loop** — shows real understanding
9. **Python GIL** — any Python shop
10. **CAS and ABA problem** — senior/staff level lock-free discussions

End of Concurrency & Multithreading Study Notes Estimated study time: 3-4 hours for deep read, 10 minutes for cheat sheet revision

15 Visual Reference

A set of figures distilling the key quantitative intuitions of this topic.

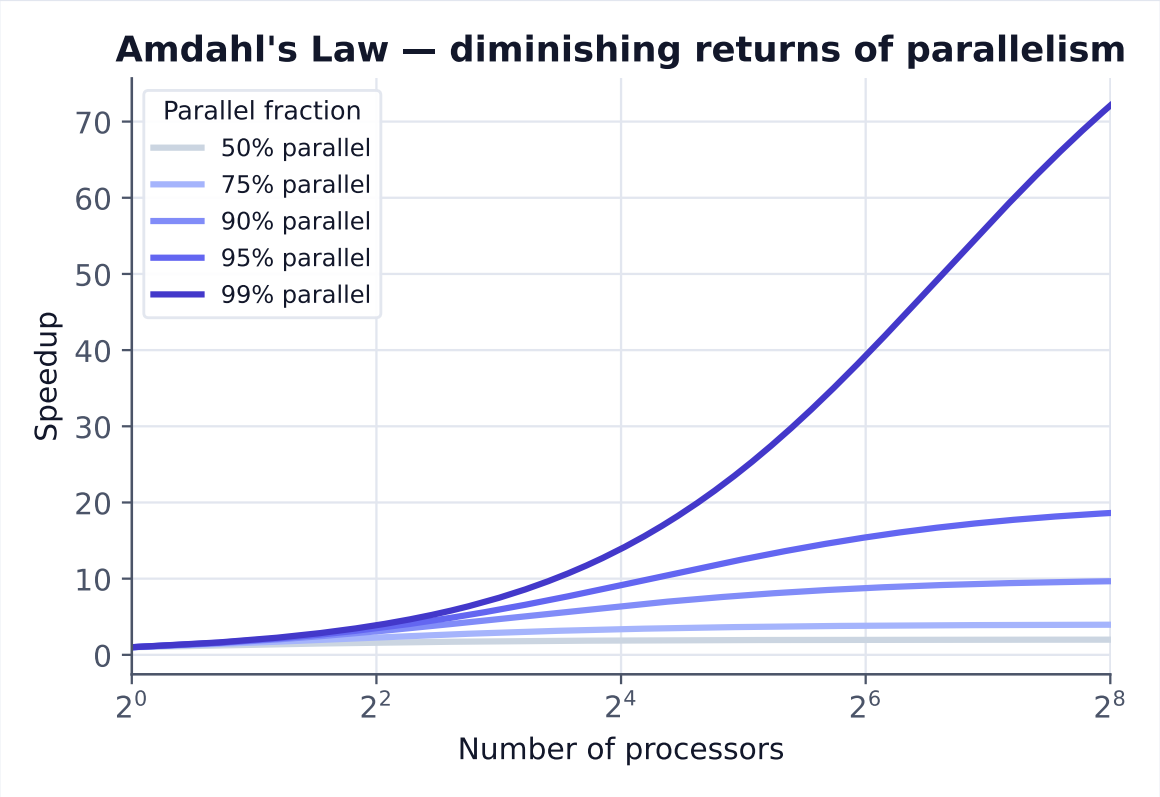


Figure 1. Amdahl's Law: maximum speedup is capped by the serial fraction. Even at 95% parallel, 256 cores yield only 17x — the serial 5% dominates.

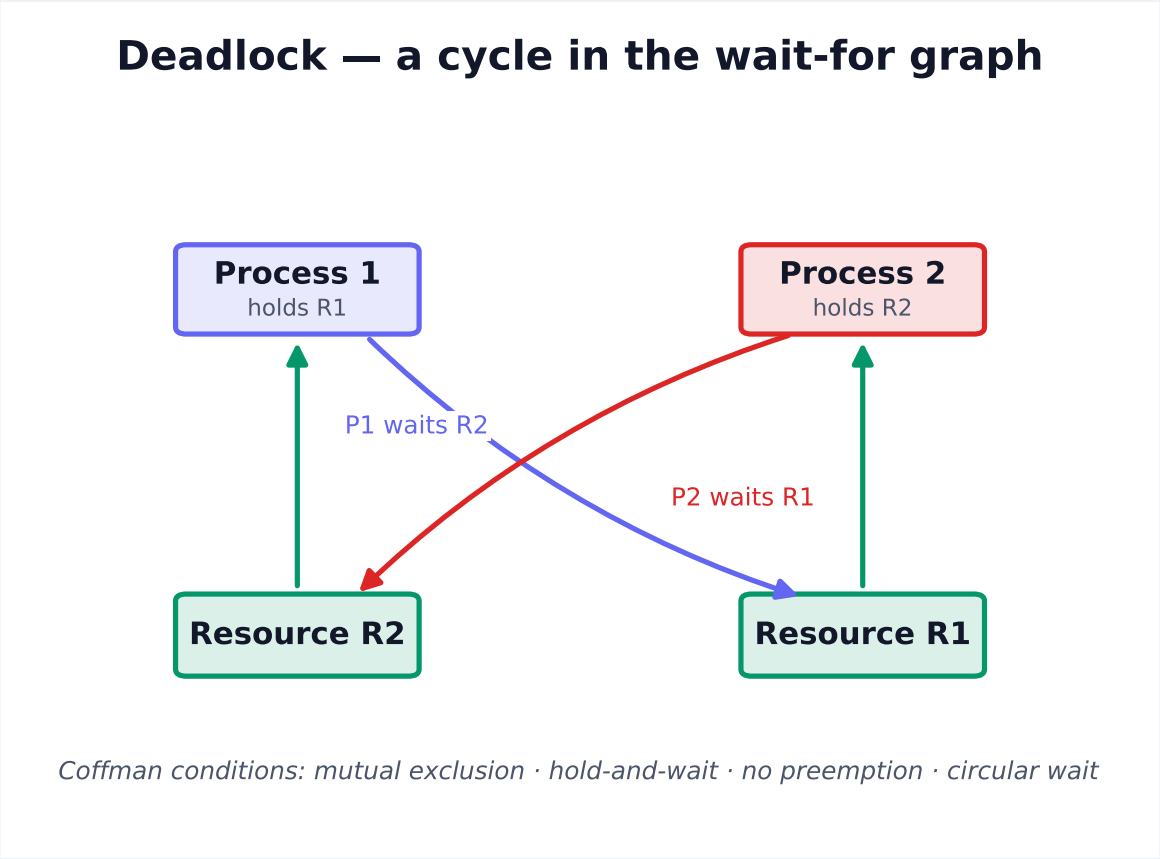


Figure 2. Deadlock needs all four Coffman conditions at once; the tell-tale sign is a cycle in the wait-for graph. Break any one condition (e.g. lock ordering) to prevent it.